Assignment #03 - AI-assisted coding, vibe coding and prototype cleanup

Smart Campus Room Availability Checker

Student	Myrzabayev Ruslan
Course	Software Engineering (Lessons 05-06)
Submitted	24 October 2026
Package	this report, `booking.py`, `test_booking.py`, `AI_Evidence.pdf` (appendix), Bolt.new project + demo + repository (links in section 10)

> Following the feedback on Assignment 2, all original prompts/responses, original screenshots and diagram files, AI tool/model/date, references and source checks are in the separate appendix **AI_Evidence.pdf**. This report only references them: E-xx = exchange, S-xx = screenshot of the A3 project, P-xx = screenshot of my earlier Practice 6 project.

1. Task 1 - Baseline and plan (written before prompting)

1.1 Scope in my own words

One responsive page. The student picks one room, sets a (simulated) current time and a start/end time, presses a button and gets **Available** or **Unavailable** with a reason. All rooms and bookings are fictional and live in memory. The page only *checks*; it never creates, confirms or edits a booking. Out of scope: login, database, backend, external API, payments, anything not related to checking.

1.2 Acceptance criteria in my own words

AC	My wording
AC1	Times are whole minutes from midnight. Valid only if 0 ≤ start < end ≤ 1440 and start is strictly later than now (start == now is rejected).
AC2	Length end - start can be at most 120. 120 is fine, 121 is not.
AC3	If the room is blocked, the answer is always Unavailable, whatever the times are.
AC4	No overlap with any existing booking. Intervals are half-open, so 600-660 and 660-720 just touch and are OK.
AC5	The result is clearly one of two states, it belongs to the inputs on the screen, and checking never changes the booking data.
AC6	Errors are explained in normal words, and the page works on a narrow phone screen.

1.3 Likely failure points (my prediction)

1. **Endpoint overlap** - using <= instead of < makes touching bookings "overlap" (660-720 wrongly rejected).
2. **Starts-now** - using `start < now` instead of `start <= now` accepts 540-570 when now = 540.
3. **Exactly 120** - using >= rejects a valid 2-hour slot.
4. **UI cannot express the rule** - a time picker with big steps (e.g. 15 min) makes 14:01 or 24:00 impossible to enter, so some tests can't even be run.

5. **Only the first booking is checked** - a loop that returns too early.
6. **Non-integer input** - Python accepts 660.5 or `True` silently because nothing checks the type.

1.4 Function contract

```
can_book(start, end, now, blocked, existing) -> bool
  start, end : int, minutes after midnight (candidate, half-open [start, end))
  now        : int, 0..1439 (assumed valid)
  blocked    : bool, True = selected room is blocked
  existing   : list of (s, e) int tuples, read-only
Returns True  only if AC1-AC4 all hold.
Returns False otherwise (never raises for bad times, never mutates inputs).
```

1.5 Initial assumptions and what I verify manually

- Assumption: `now` is valid (given by the task); existing bookings are already valid and may be unsorted.
- Assumption: bool is not a valid minute even though Python treats `True` as 1.
- Manual checks: I read every branch of the generated code myself, run the tests myself (never trusting "build passes" or "all tests pass" from the AI), click through the Task 4 cases in the Bolt preview, and look at the page in a phone-size preview.

2. Task 2 - AI-assisted Python implementation

Files: `booking.py` (final), `v1/booking.py` (first AI output, saved unchanged), `test_booking.py` (21 unittest cases, standard library only). The same test file runs both versions: `python3 -m unittest -v test_booking` and `BOOKING_IMPL=v1 python3 -m unittest -v test_booking`. AI tool, prompt and raw output: E-P1, E-P2.

2.1 Branch review (final version)

Branch in `can_book`	Why it satisfies the AC
`if blocked: return False`	AC3 - first check, so every request for a blocked room is rejected.
`if not (_is_minute(start) and _is_minute(end))`	AC1 "integer minutes". `_is_minute` excludes bool on purpose. Added after v1 (issue P-01).
`if not (0 <= start < end <= 1440)`	AC1 day bounds, rejects reversed and zero-length intervals in one chained comparison.
`if start <= now`	AC1 "strictly after now" - `<=` is what rejects starts-now.
`if end - start > 120`	AC2 - strict `>` so exactly 120 passes.
`for s, e in existing: if start < e and s < end`	AC4 - standard half-open overlap test; touching is not overlap. Loops through *all* bookings.
`return True` + no writes anywhere	AC5 - returns a real bool, `existing` is only iterated (checked by T19).

v1 had the same structure without the type check and without named constants/docstring. That is the only behaviour difference (see 4.1).

3. Task 3 - Prototype v1 with Bolt.new

For A3 I started a **new** Bolt.new project (`sb1-gewmu6gh`) so v1 is a clean first generation for this assignment. The exact prompt is E-B1 (S-01). It lists AC1-AC6 with the A3 wording, says to treat them as fixed, and explicitly forbids **login/authentication, database (no Supabase or storage), backend/server**, external APIs, payments and analytics. Tool: Bolt.new, mode "Bolt Agent - Standard"; the underlying LLM name is **not displayed**, so I don't claim one. Date: 24 Oct 2026, Version 1 at 22:46. First usable preview: S-02.

(Earlier, in Practice 6, I built a similar prototype in another Bolt project, `sb1-svxv4k4n`, and cleaned it up in 7 versions. That work is kept in appendix F as history; it is not the A3 v1.)

3.1 Main generated files and their role (I opened each in the Code view)

File	Role
`src/lib/availability.ts`	Pure function `checkAvailability({now, start, end, room})` returning `{available, code, reason}`; types `Booking`, `Room`; helper `fmt()` for HH:MM. AC1 check includes `Number.isInteger` on start/end/now (S-07).
`src/data/rooms.ts`	4 fictional rooms: Athena Library Room 201 (booking 10:00-11:00 = 600-660, the Task 4 base data), Tesla Engineering Lab 104 (2 bookings), Curie Studio 007 (none), Newton Lecture Hall A (blocked).
`src/components/RoomPicker.tsx`	Room list with radio selection and booking count / Blocked label.
`src/components/TimeSelect.tsx`	Dropdown for now/start/end. In v1 it only offered 15-minute steps (S-06).
`src/components/ResultPanel.tsx`	Green Available / red Unavailable panel with the reason.
`src/App.tsx`	Page layout, state, inline validation hints, Check/Reset buttons.

v1 was **not** treated as final. The logic file looked right, but the UI around it broke AC2 testing and AC5 (section 5.2).

4. Task 4 - Test execution

4.1 Python - executed 24 Oct 2026, Python 3.13

ID	Case (now = 540, existing = [(600,660)] unless noted)	Expected	v1	Final
T01	660-720 touching	True	True	True
T02	630-690 overlap	False	False	False
T03	660-720, blocked=True	False	False	False
T04	720-840 exactly 120	True	True	True
T05	720-841 121 min	False	False	False
T06	540-570 starts now	False	False	False
T07	720-660 reversed	False	False	False
T08	700-700 zero length	False	False	False
T09	1380-1440 end of day	True	True	True
T10	1400-1441 past day	False	False	False

T11	-10 to 30 (now 0, no bookings)	False	False	False
T12	500-560 start in past	False	False	False
T13	541-571 one minute after now	True	True	True
T14	560-600 touching start of booking	True	True	True
T15	610-650 inside booking	False	False	False
T16	590-670 covers booking	False	False	False
T17	660-720, 3 bookings, gap fits	True	True	True
T18	850-910 hits 3rd booking (900-960)	False	False	False
T19	inputs unchanged after 2 calls (unsorted list)	equal	equal	equal
T20	660.5-720 / 660-720.0 / True-60	False	True / True / True	False
T21	return type is exactly bool	bool	bool	bool
Total			**20/21**	**21/21**

Raw outputs: `v1_test_run.txt`, `v1_type_probe.txt`, `final_test_run.txt` (appendix C).

4.2 Bolt prototype - manual UI runs (24 Oct 2026)

Setup: Athena Library Room 201 (booking 10:00-11:00 = 600-660), Simulated now 09:00 (= 540). Blocked room: Newton Lecture Hall A. v1 = Version 1 (22:46), final = Version 4 (22:53).

Task 4 case	UI input	Expected	v1 actual	Final actual
Touching 660-720	11:00-12:00	Available	Available (S-03)	Available (S-22)
Overlap 630-690	10:30-11:30	Unavailable	Unavailable, "Overlaps an existing booking (10:00-11:00)" (S-04)	same (S-23)
Blocked	Newton, valid time	Unavailable	after an earlier check, a stale "**Available**" panel was shown for the blocked room (S-10)	Unavailable, "Newton Lecture Hall A is currently blocked" (S-26); Vitest A03 660-720 pass
Exactly 120: 720-840	12:00-14:00	Available	Available (S-05)	Vitest A04 pass (UI not repeated)
121 min: 720-841	12:00-14:01	Unavailable	**Could not be entered** - only 15-min steps (S-06)	Unavailable, "Booking length is 121 min. Maximum allowed is 120 min." (S-24)
Starts now 540-570	09:00-09:30	Unavailable	No result at all: Check button disabled, only a small hint (S-09)	Unavailable, "Start time must be strictly later..." (S-21)
Blocked + starts now	Newton 09:00-09:30	Unavailable, reason = blocked	v3: Unavailable, reason = time error (S-14)	Vitest A15 pass (reason matches /blocked/)
Result belongs to inputs	check, then change room	old result gone	old result stays (S-10)	result cleared (S-25)
Mobile layout	iPhone 17 preview 402×874	usable	-	single column, readable (S-27)

4.3 Bolt prototype - automated acceptance set on the final version

To rerun the *complete* set (including reversed times, day bounds, multiple bookings and unchanged input), I added `src/lib/availability.test.ts` with 15 Vitest cases A01-A15 (drafted with Claude, E-C1; written into the project from the Bolt terminal, not by a Bolt prompt, to save the free-plan tokens). I installed `vitest@2.1.9` and ran `npx vitest run` myself: **15 passed** at 22:56:32 (S-19, S-20). Each test also checks that the room object is unchanged (AC5). `npm run build` passed before that (S-18). These tests were only run on the final version, not on v1.

5. Evidence, traceability and cleanup (Task 5)

5.1 Traceability table

v1 actual = Python v1 / Bolt Version 1; final actual = Python final / Bolt Version 4.

AC	Implementation / file	Test input	Expected	v1 actual	Final actual
AC1	`booking.py` bounds + `start <= now`; `availability.ts` AC1 block	540-570; 720-660; 1380-1440; 1400-1441	F; F; T; F	Py: F; F; T; F. Bolt UI 540-570: no result shown	Py: same. Bolt: F; F; T; F (A06, A07, A09, A10; S-21)
AC1 (int)	`_is_minute()`; `Number.isInteger`	660.5-720; True-60	F; F	Py: T; T	Py: F; F. Bolt A12: F
AC2	`end - start > 120`; `TimeSelect.tsx` options	720-840; 720-841	T; F	Py: T; F. Bolt: T; 841 not enterable	T; F (A04, A05, S-24)
AC3	`if blocked` first (Python); moved to top in `availability.ts` (v4)	660-720 blocked; 540-570 blocked	F, reason "blocked"	Py: F. Bolt UI: stale "Available"; v3 reason = time	F, reason "blocked" (A03, A15, S-26)
AC4	half-open check over all bookings	660-720; 630-690; 850-910 with 3 bookings	T; F; F	Py: T; F; F. Bolt UI: T; F	T; F; F (A01, A02, A14)
AC5	no writes; result cleared on input change (`App.tsx` v2)	deep compare; change room after a check	unchanged; old result gone	data unchanged; old result stays	unchanged (all A-tests); cleared (S-25)
AC6	`App.tsx` layout; reason texts	phone preview; blocked + bad time	readable; reason "blocked"	reason = time error (S-14)	"...is currently blocked..." (S-26); mobile OK (S-27)

5.2 Issue log

ID	Issue	Reproduction (evidence)	Priority	Prompt / change	Retest result
V1-01	Functional (AC1/AC3/AC5): stale result. After one check the result panel stays when room/time change and even shows "Available - free for 09:00-09:30" for the **blocked** room with start = now. Also starts-now never gives an Unavailable result (Check disabled).	Athena 12:00-14:00 Check → switch to Newton, set 09:00-09:30 (S-10, S-09)	High	E-B2 → `App.tsx`: clear result on any input change; Check always enabled, always shows the result (v2)	Result cleared (S-25); starts-now → Unavailable (S-21)

V1-02	Functional/testability (AC1/AC2): 15-minute steps, 12:00-14:01 cannot be entered	`TimeSelect.tsx` line 14 `i * 15` (S-06)	High	E-B3 → 1441 options, every minute 00:00-24:00 (v3, S-17)	12:00-14:01 → Unavailable 121 min (S-24)
V1-03	Usability (AC3/AC6): blocked room reported with a time error	Newton, now 09:00, 09:00-09:30 → "Start time must be strictly later..." (S-14; code S-08)	Medium	E-B4 → move `if (room.blocked)` to the top (v4, S-16)	Reason = blocked (S-26, Vitest A15)
P-01	Functional (AC1): Python v1 accepts non-integer minutes	`can_book(660.5, 720, 540, False, [(600,660)])` → True	High	E-P2 → `_is_minute()` guard, constants, docstring	21/21, T20 passes

Every fix was a small prompt about one issue with the exact reproduction, the file allowed to change and what must *not* change. No "make it better" prompts and no new features. **Before/after screenshots:** V1-01 S-10 → S-25; V1-02 S-06 → S-17/S-24; V1-03 S-14 → S-26.

6. Task 6 - Coding-agent analysis

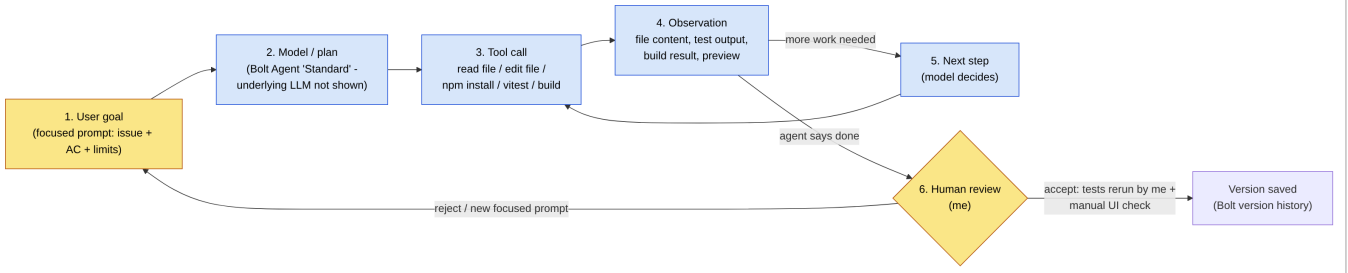


Figure 1. Agent loop as I used it (source: `diagrams/agent_loop.mmd`, original PNG in appendix).

6.1 Context, tools and permissions

	Observed (I saw it)	Assumed (not visible to me)
Bolt.new context	My prompt, the chat history, project files - replies show "files read", "Read src/App.tsx" before edits; Bolt said it was "reviewing the existing project files" in v1	How much history is really sent to the model each turn; which LLM is behind "Standard"
Bolt.new tools	File read/edit ("actions taken"), shell (build) in a browser WebContainer, live preview. In V1-03 it first added the block on top and then removed the old copy in a second step	Exact tool API / harness and how retries are decided
Bolt.new permissions	Edits any project file and runs commands **without asking me**; free plan shows a daily token limit (S-15). It can't publish or push to GitHub unless I click. Version history lets me roll back	Network access of the WebContainer beyond npm
Claude (Python part, tests)	Ran Python and the tests in a sandbox, drafted the Vitest file, drove my browser to run Bolt and the UI checks	Internal planning of the model

Observed vs claimed: Bolt's v1 summary says the logic file is "the single source of truth for all six acceptance criteria" and "validation messages guide the student ... before they even press Check". The logic file was fine, but that same "guide before Check" design disabled the button and left an old result on screen (V1-01). The claim was about one file; the bug was in another.

6.2 Safety checkpoint

Before I accept an AI change I always do the same checkpoint: **read the changed code + run the build/tests myself + repeat the original reproduction in the UI**. If the agent touched a file I did not allow, or the tests were changed to match the code, I reject it and roll back to the previous version. For V1-03 this meant checking that the blocked block is now at lines 47-53 and the old copy after AC2 is gone (S-16), then `npm run build` (S-18), the Vitest run (S-20) and the UI (S-26).

6.3 Reliability, security and human responsibility

Reliability: the agent writes a lot of correct code quickly, but its summary sounds sure even when the UI is wrong; one stale state bug was enough to show "Available" for a blocked room. **Security:** an agent with shell + npm access can pull any package into the project and edit any file. In a real system those changes need review and limited permissions. **Human responsibility:** I fixed the AC, I decide what counts as a pass, and I submit the work, so I'm responsible for every line and every "pass" claim.

7. Reflection - dependability and AI

In the Python part I used AI-assisted coding: I wrote the contract and tests first, took the generated v1, ran it and found one real failure (non-integer minutes). In Bolt it was vibe coding: one prompt gave a full app, and the logic file even looked correct, but the problems only appeared when I clicked through it. Reliability risk: a stale result made the page say "Available" for a blocked room. Availability/usability risk: 15-minute steps meant a valid input like 14:01 could not be entered at all. Security risk: the agent edits any file and runs npm commands without asking. That's why code review, tool permissions and tests stay my job. The AI only follows the prompt and doesn't know which behaviour is right. I rejected Bolt's design of disabling Check for invalid input: it looked friendly, but it hid the Unavailable result that AC5 asks for.

(Reflection: 147 words.)

Next improvements (same scope): add a UI test for "result clears when inputs change"; limit Simulated now to 00:00-23:59 (now = 24:00 is selectable today); show the Blocked state next to the time form too.

8. Remaining limitations

- "Simulated now" uses the same picker as End, so 24:00 can be chosen as now. The task assumes now is 0-1439, so I didn't change it (out of the fixed scope), but noted it.
- Vitest was run on the final version only. v1 results come from manual UI runs and code reading.
- UI tested in the Bolt preview (desktop and iPhone 17 device preview), not on a real phone.
- The Bolt free plan has a daily token limit, so I kept prompts to the 3 needed fixes.

9. References

1. Lesson #05 - Software Engineering - Lecture: dependable systems, security, reuse.
2. Lesson #05 - Practice: AI-assisted coding and `can_book` tests.
3. Lesson #06 - Lecture: coding agents, agent loop, tools and harness design.
4. Lesson #06 - Practice: vibe coding, Bolt.new and prototype cleanup.
5. Assignment #02 format and evidence-driven AI workflow (baseline).
6. Python 3 docs - `unittest`; `bool` is a subclass of `int`.
7. Vitest documentation - `vitest run`.

10. Submission links

Bolt.new project (A3)	https://bolt.new/~/sb1-gewmu6gh
Earlier Practice 6 project	https://bolt.new/~/sb1-svxv4k4n
Working demo	https://smart-campus-room-av-kyfa.bolt.host (S-28)
Repository	https://github.com/Rusikkooo/smart-campus-room-checker